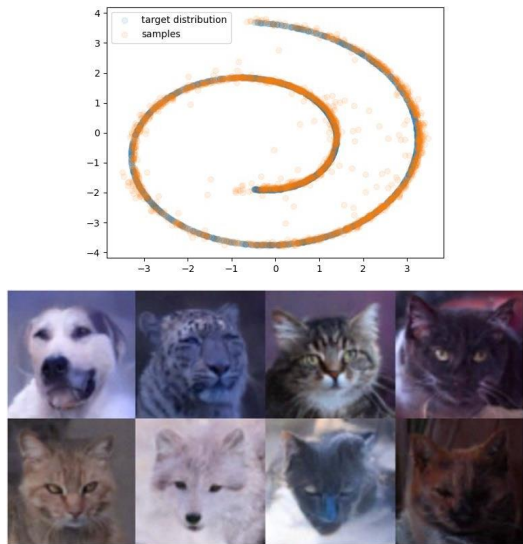
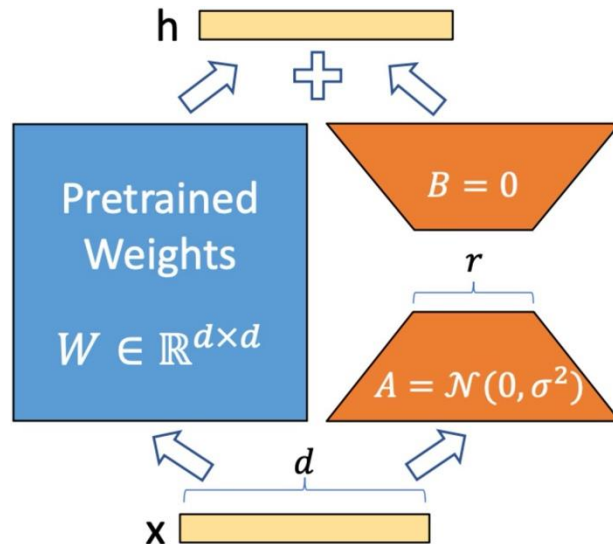


Lab2 - DDIM and LoRA



Task 1 - DDIM

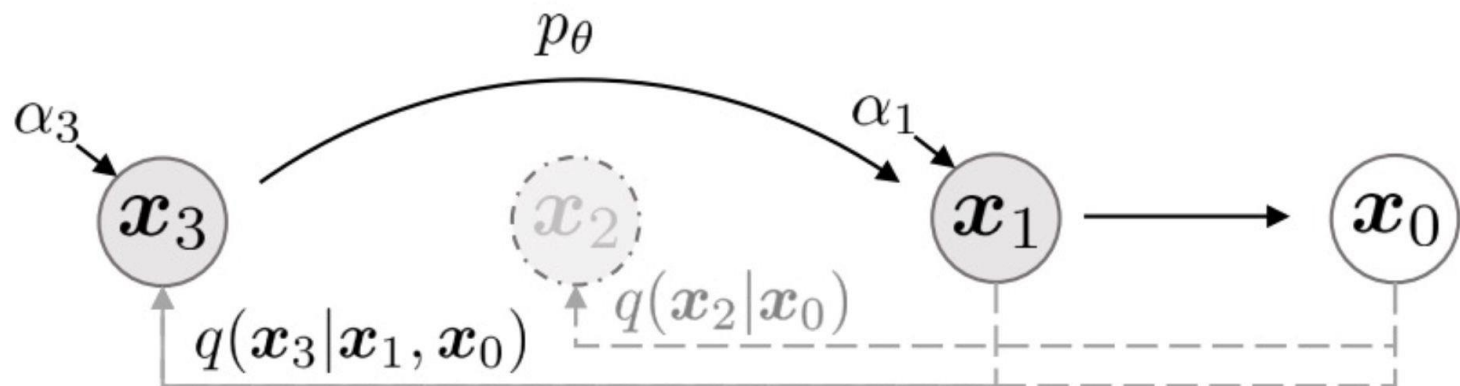


Task 2 - LoRA

[Lab2 Code \(download\)](#)

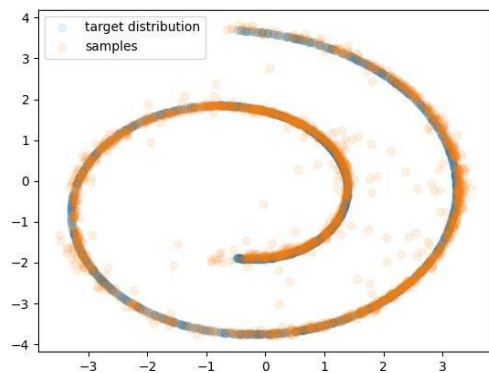
Introduction - Task 1

Implement Denoising Diffusion Implicit Models (DDIM) to accelerate the generation process of pretrained DDPM.



[Denoising Diffusion Implicit Models, Song et al., ICLR 2021](#)

Introduction - Task 1



You need to implement:

- **Task 1-1:** [2D Swiss Roll] Reverse Process of DDIMs
- **Task 1-2:** [Image Generation] DDIM Sampling

[Denoising Diffusion Implicit Models, Song et al., ICLR 2021](#)

TODO - Task 1-1 : DDIM [2D Swiss Roll]

#TODO1 - implement `ddim_p_sample`

- `2d_plot_diffusion_todo/ddpm.py`

#TODO2 - implement `ddim_p_sample_loop`

- `2d_plot_diffusion_todo/ddpm.py`

#TODO3 - run the evaluation

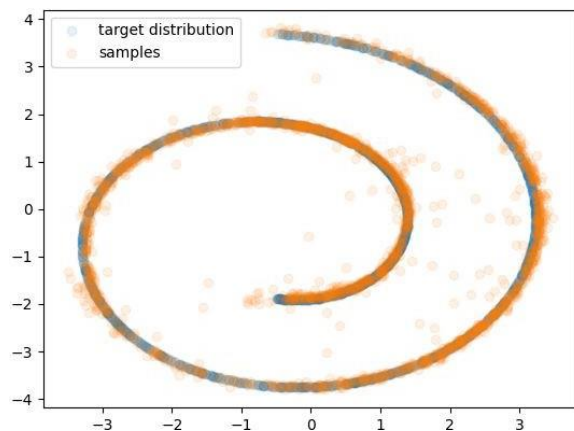
- `2d_plot_diffusion_todo/ddpm_tutorial.ipynb`

Run all cells in `2d_plot_diffusion_todo/ddpm_tutorial.ipynb` and generate 2D points via DDIM sampling.

TODO - Task 1-1 : DDIM [2D Swiss Roll]

#Report

1. Complete and Explain of all #TODO code (15pts)
2. Fig of evaluation result (2.5 pts)



3. A screenshot of the Chamfer Distance measured using DDIM (2.5 pts if < 40)

TODO - Task 1-2 : DDIM [Image Generation]

#TODO1 - implement `set_inference_timesteps` in `DDIMScheduler`

- `image_diffusion_todo/scheduler.py`

#TODO2 - implement `step` in `DDIMScheduler`

- `image_diffusion_todo/scheduler.py`

Only implement DDIM with noise predictor, no `x0` or mean predictor implementation is required.

#TODO3 - run the sampling

- `image_diffusion_todo/sampling.py`

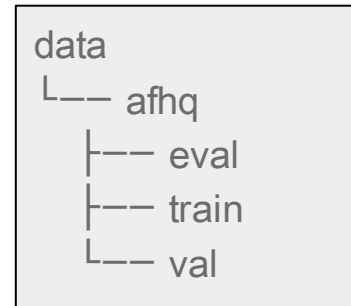
```
python sampling.py --ckpt_path {CKPT} --save_dir {SAVE_DIR}
--sample_method ddim --ddim_steps {DDIM_STEPS} --eta {ETA}
```

- Try different **DDIM_STEPS**: 10, 20, 50, 100, 1000
- Try different **ETA**: 0.0, 0.2, 0.5, 1.0

TODO - Task 1-2 : DDIM [Image Generation]

#TODO4 – evaluate FID

- image_diffusion_todo/dataset.py
- image_diffusion_todo/fid/measure_fid.py



STEP 1: Run `python dataset.py` once to prepare the evaluation data.

This will create the eval directory under `data/afhq`.

Do NOT forget to run this. Otherwise, you will get incorrect FIDs!

STEP 2: Run the following command to evaluate FID:

```
python fid/measure_fid.py data/afhq/eval/ {SAMPLE_DIR}
```

TODO - Task 1-2 : DDIM [Image Generation]

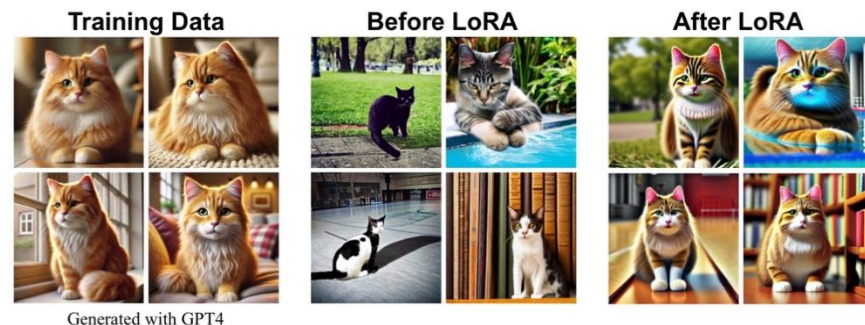
#Report

1. Complete and Explain of all #TODO code (20pts)
2. Provide a table of evaluation results. (10 pts)

FID		S				
		10	20	50	100	1000
η	0.0	18.24	18.66	21.77	25.55	25.30
	0.2	12.40	16.04	13.91	17.35	19.18
	0.5	16.35	10.48	8.05	6.81	5.80
	1.0	29.18	20.02	9.36	6.27	4.73

3. Discuss and explain the evaluation results. (10 pts)

Introduction - Task 2



You need to implement:

- **Task 2-1:** Train LoRA on a specific style
- **Task 2-2:** Train DreamBooth with LoRA on a specific identity

[LoRA: Low-Rank Adaptation of Large Language Models, Hu et al., ICLR 2022](#)

Introduction to Hugging Face and Diffusers

Hugging Face 🤗

An open-source platform that serves as a hub for machine learning applications.

The screenshot shows the Hugging Face homepage. At the top, there's a navigation bar with the Hugging Face logo, a search bar, and tabs for Models, Datasets, Spaces, Posts, Docs, and Pricing. The 'Models' tab is highlighted with a red box. Below the navigation bar, the main content area is divided into three sections. On the left, there's a sidebar for the user 'phillipinseoul' with links to Profile, Inbox (0), Settings, Billing, and Get Pro. The middle section is titled 'Following' and shows a list of followed creators: 'as-cle-bert' (Bioinformatics ML), 'bwang0911' (Research on sentence embeddi...), and '1littlecoder' (Great YouTube channel and cod...). On the right, there's a 'Trending' section for the last 7 days, featuring models like 'openai/whisper-large-v3-tu...' (Automatic Speech Recognition) and 'nvidia/NVLM-D-72B' (Image-Text-to-Text). A prominent purple button for 'Open NotebookLM' is also visible.

Diffusers Library ffusers

```
import torch
from diffusers import StableDiffusionPipeline

model_id = "runwayml/stable-diffusion-v1-5"
pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
    torch_dtype=torch.float16
)
pipe = pipe.to("cuda")

prompt = "a photo of an astronaut riding a horse on mars"
image = pipe(prompt).images[0]

image.save("astronaut_rides_horse.png")
```

e.g. Loading Stable Diffusion from diffusers

Popular Tasks & Pipelines

Task	Pipeline	 Hub
Unconditional Image Generation	DDPM	google/ddpm-ema-church-256
Text-to-Image	Stable Diffusion Text-to-Image	runwayml/stable-diffusion-v1-5
Text-to-Image	unclip	kakaobrain/karlo-v1-alpha
Text-to-Image	DeepFloyd IF	DeepFloyd/IF-I-XL-v1.0
Text-to-Image	Kandinsky	kandinsky-community/kandinsky-2-2-decoder
Text-guided Image-to-Image	Controlnet	llyasviel/sd-controlnet-canny
Text-guided Image-to-Image	Instruct Pix2Pix	timbrooks/instruct-pix2pix
Text-guided Image-to-Image	Stable Diffusion Image-to-Image	runwayml/stable-diffusion-v1-5
Text-guided Image Inpainting	Stable Diffusion Inpaint	runwayml/stable-diffusion-inpainting
Image Variation	Stable Diffusion Image Variation	lambdalabs/sd-image-variations-diffusers
Super Resolution	Stable Diffusion Upscale	stabilityai/stable-diffusion-x4-upscaler
Super Resolution	Stable Diffusion Latent Upscale	stabilityai/sd-x2-latent-upscaler

TODO - Task 2: Training Custom LoRA Models

#TODO0

You need access to Hugging Face to proceed with Task 2 :

1. Sign into Hugging Face.
2. Obtain your access token at <https://huggingface.co/settings/tokens>.
3. From your terminal, log into Hugging Face using

```
$ hf auth login
```

and enter your Access Token.

TODO - Task 2: Training Custom LoRA Models

#TODO0

4. To check the access to Hugging Face, download Stable Diffusion from Hugging Face and generate an image with it:

```
import torch
from diffusers import StableDiffusionPipeline

model_id = "CompVis/stable-diffusion-v1-4"
device = "cuda"

pipe = StableDiffusionPipeline.from_pretrained(model_id, torch_dtype=torch.float16)
pipe = pipe.to(device)

prompt = "a photo of an astronaut riding a horse on mars"
image = pipe(prompt).images[0]

image.save("astronaut_rides_horse.png")
```

TODO - Task 2-1: Train LoRA on a specific style

#TODO1 - train

You can either use an open-source dataset and train with

```
$ sh scripts/train_lora.sh
```

Or create a custom dataset and train with

```
$ sh scripts/train_lora_custom.sh
```

```
##### BEGIN TODO: experiment configuration #####
# Configuration only: these defaults are complete;
# no implementation is required.
export MODEL_NAME="CompVis/stable-diffusion-v1-4"
export DATASET_NAME="lamdalabs/naruto-blip-captions"
DATASET_CONFIG_NAME=""
export OUTPUT_DIR="./runs/sd-naruto-model-lora"

...

SEED=42
CHECKPOINTS_TOTAL_LIMIT=2
VALIDATION_PROMPT="a cute bear"
##### END TODO: experiment configuration #####
```

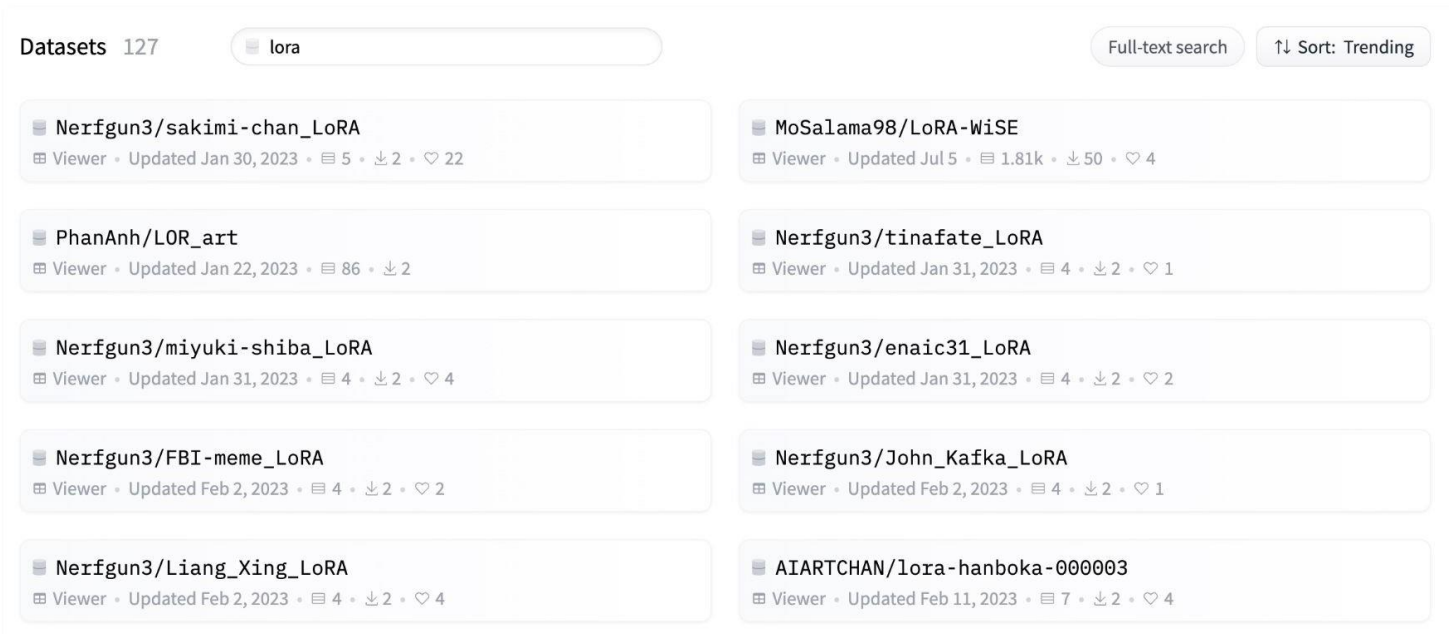
```
##### BEGIN TODO: experiment configuration #####
# Configuration only: these defaults are complete;
# no implementation is required.
export MODEL_NAME="CompVis/stable-diffusion-v1-4"
export TRAIN_DATA_DIR="./sample_data/artistic-custom"
export OUTPUT_DIR="./runs/artistic_custom"
MIXED_PRECISION="no"
IMAGE_COLUMN="image"

...

SEED=42
CHECKPOINTS_TOTAL_LIMIT=2
VALIDATION_PROMPT="a house"
##### END TODO: experiment configuration #####
```

TODO - Task 2-1: Train LoRA on a specific style

TIP: You can find many open-source LoRA training datasets on Hugging Face.



The screenshot displays the Hugging Face Datasets interface. At the top, there's a search bar with 'lora' entered, and buttons for 'Full-text search' and 'Sort: Trending'. Below the search bar, a grid of dataset cards is shown. Each card includes the dataset name, a 'Viewer' link, the update date, and statistics for discussions, downloads, and likes.

Dataset Name	Viewer	Updated	Discussions	Downloads	Likes
Nerfgun3/sakimi-chan_LoRA	Viewer	Jan 30, 2023	5	2	22
MoSalama98/LoRA-WiSE	Viewer	Jul 5	1.81k	50	4
PhanAnh/LOR_art	Viewer	Jan 22, 2023	86	2	
Nerfgun3/tinafate_LoRA	Viewer	Jan 31, 2023	4	2	1
Nerfgun3/miyuki-shiba_LoRA	Viewer	Jan 31, 2023	4	2	4
Nerfgun3/enaic31_LoRA	Viewer	Jan 31, 2023	4	2	2
Nerfgun3/FBI-meme_LoRA	Viewer	Feb 2, 2023	4	2	2
Nerfgun3/John_Kafka_LoRA	Viewer	Feb 2, 2023	4	2	1
Nerfgun3/Liang_Xing_LoRA	Viewer	Feb 2, 2023	4	2	4
AIARTCHAN/lora-hanboka-000003	Viewer	Feb 11, 2023	7	2	4

TODO - Task 2-2: Train LoRA on a specific identity

#TODO1 - train

Run `$ sh scripts/train_dreambooth_lora.sh` for training.

```
##### BEGIN TODO: experiment configuration #####
# Configuration only: these defaults are complete;
# no implementation is required.
export MODEL_NAME="CompVis/stable-diffusion-v1-4"
export INSTANCE_DIR="./sample_data/dreambooth-cat"
export OUTPUT_DIR="./runs/dreambooth_cat"
INSTANCE_PROMPT="a photo of a sks cat"

...

LR_WARMUP_STEPS=0
MAX_TRAIN_STEPS=500
VALIDATION_PROMPT="A photo of sks cat in a bucket"
VALIDATION_EPOCHS=50
CHECKPOINTS_TOTAL_LIMIT=2
SEED=0
##### END TODO: experiment configuration #####
```

TODO - Task 2: Training Custom LoRA Models

#TODO2 - inference

- task2/lora_inference.ipynb

```
##### BEGIN TODO: experiment configuration #####
# Configuration only: these defaults are complete; no implementation is required.
model_name = "CompVis/stable-diffusion-v1-4"
# Match this path to the chosen training script's OUTPUT_DIR.
# Other examples: "./runs/sd-naruto-model-lora", "./runs/dreambooth_cat".
# Set lora_path to None (or "") to generate with the base model only.
lora_path = "./runs/artistic_custom"
prompt = "a man with sunglasses"
seed = 10
device = "cuda:0"
torch_dtype = torch.float16
num_inference_steps = 30
guidance_scale = 7.5
##### END TODO: experiment configuration #####
```

TODO - Task 2: Training Custom LoRA Models

#Report

1. [Task 2-1] Description on the dataset used, including its source. (5 pts)
2. [Task 2-1] Visualization of training and generated images with the corresponding text prompts. (15 pts)
3. [Task 2-2] Description on the dataset used, including its source. (5 pts)
4. [Task 2-2] Visualization of training and generated images with the corresponding text prompts. (15 pts)

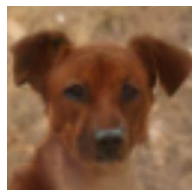
Submit

Create and zip a single folder named {ID}_lab2.zip including:

- The PDF file formatted following the guideline
- **(Task1)** The complete code for Task 1, excluding checkpoints and datasets.
- **(Task2)** Provide the LoRA weight cloud link in a text file.

```
{ID}_task2-1_lora_weight.txt
```

```
{ID}_task2-2_lora_weight.txt
```



Thank you

```
{ID}_lab2.zip
├── report.pdf
├── task1
│   ├── 2d_plot_diffusion_todo
│   │   ├── __init__.py
│   │   ├── chamferdist.py
│   │   ├── dataset.py
│   │   ├── ddpm_tutorial.ipynb
│   │   ├── ddpm.py
│   │   └── network.py
│   └── image_diffusion_todo
│       ├── dataset.py
│       ├── model.py
│       ├── module.py
│       ├── network.py
│       ├── sampling.py
│       ├── scheduler.py
│       └── train.py
└── task2
    ├── {ID}_task2-1_lora_weight.txt
    └── {ID}_task2-2_lora_weight.txt
```